**Fundamentos de Sistemas Computacionales**
**Proyecto I**

Profesor: Milton Villegas Lemus
2 de Noviembre de 2026

TEC | Tecnológico de Costa Rica

juanmaboniarce@gmail.com
randallmz7923@gmail.com

# Carro de juguete controlado por comandos con Raspberry Pi Pico W

Juan Manuel Bonilla Arce, carné 2024105762
Randall Andrés Mora Zamora, carné 2023344786
Escuela de Ingeniería en Computadores, Instituto Tecnológico de Costa Rica

Palabras clave: `telemetría, Raspberry Pi Pico W, MicroPython, PWM, L298N, 74LS164`

---

# Índice

**1. Introducción** **2**
1.1. Objetivos . . . . . 2

**2. Conclusiones** **2**

**3. Recomendaciones** **3**

**4. Análisis de resultados** **3**
4.1. Diagramas de módulos . . . . . 3
4.1.1. Módulos de hardware . . . . . 4
4.1.2. Módulos de software . . . . . 5
4.2. Plan de pruebas . . . . . 5
4.3. Reglas de grupo . . . . . 7
4.4. Roles y fechas de actividades y entregables . . . . . 7

**5. Bitácora** **8**

# 1. Introducción

La telemetría es medir un objeto a distancia y mandar esos datos a quien controla el sistema. En la Fórmula 1 se usa desde 1975. Hoy los equipos la utilizan para medir, controlar y ajustar el carro mientras está en la pista. La Fórmula E heredó la misma idea para los carros eléctricos.

En este proyecto se aplica esa idea a un carro de juguete. El carro recibe comandos escritos en la terminal de Thonny desde la computadora. Una Raspberry Pi Pico W los interpreta y genera las señales para el resto del circuito.

El carro tiene dos motores. Uno da la tracción, hacia adelante y hacia atrás. El otro da la dirección, a la izquierda y a la derecha. Los dos se controlan con un driver L298N. La velocidad se cambia con modulación por ancho de pulso (PWM). El carro también tiene seis LEDs. Son dos luces frontales, dos traseras y dos direccionales. Se controlan con un registro de corrimiento 74LS164, que solo necesita dos pines de la Pico. Por último, una fotorresistencia mide la luz del ambiente y permite encender las luces de forma automática cuando está oscuro.

El programa está hecho en MicroPython. Acepta los comandos de la Tabla 1 del enunciado. Estos son `pwm`, `dir`, `lf`, `lb`, `ll`, `lr` y `ldr`, más los patrones `Circle`, `Infinite`, `ZigZag` y un movimiento especial.

## 1.1. Objetivos

**Objetivo general.** Construir un proyecto que integre el desarrollo de software y hardware, y aplicar conceptos básicos de proyectos propios de Ingeniería en Computadores.

**Objetivos específicos.**

- Construir un dispositivo independiente que reciba comandos de forma remota, a partir de una aplicación en el computador.
- Implementar un circuito a partir de un diagrama electrónico, elaborar un plan de pruebas y construir un módulo final.
- Desarrollar un software capaz de comunicarse con otro, mediante un protocolo previamente definido.
- Aplicar las habilidades adquiridas en los talleres.

# 2. Conclusiones

- El registro de corrimiento 74LS164 permite controlar los seis LEDs con solo dos pines de la Pico. Cada vez que se cambia una luz se manda la secuencia completa de 8 bits, porque el registro no se puede leer.
- El PWM sí cambia la potencia que recibe el motor. Pero el motor de tracción no arranca con potencias bajas. Por eso los cuatro niveles de `pwm` quedaron entre 50000 y 65535, y la diferencia de velocidad entre ellos es pequeña.

- El motor de tracción es muy pequeño para el peso del carro. Se probó dar un pulso a potencia máxima al arrancar, pero no resolvió el problema. Es una limitación del hardware y no del programa.
- Probar cada módulo por separado antes de armar todo fue clave. Así se encontró que los motores de tracción y dirección estaban conectados al revés de lo que se suponía en el código.
- LDR: El divisor de voltaje con la fotorresistencia permite que la Pico mida la luz del ambiente con el ADC. Con un umbral calibrado, el modo ldr enciende y apaga las cuatro luces sin intervención. Un error en la resistencia y provocó que la fotorresistencia no pudiera medir bien la luz del ambiente.
- Los patrones se forman combinando tracción y dirección durante tiempos fijos, sin ningún sensor que indique dónde está el carro. Por eso el recorrido depende de la fricción del piso, de la carga de las baterías y de la potencia mínima del motor, y no se repite igual en cada corrida.

# 3. Recomendaciones

- Usar un motor de tracción con más torque, o un carro más liviano. Así se podría bajar la potencia mínima y los niveles de `pwm` se notarían más.
- Medir el voltaje de las baterías antes de cada prueba. El L298N pierde cerca de 2 V internamente. Con 6 V de baterías, al motor le llegan unos 4 V, y si las baterías están gastadas llega todavía menos.
- En MicroPython usar siempre el número GP del pin y no el número físico. Confundirlos fue uno de los primeros errores.
- Probar cada interruptor con el multímetro en modo continuidad antes de soldarlo.
- Calibrar los tiempos de los patrones con el carro en el piso real donde se va a hacer la defensa, porque la fricción cambia mucho el resultado.
- Agregar histéresis al modo `ldr`, con un umbral para encender y otro más bajo para apagar, para evitar parpadeos cuando la luz ambiente está cerca del límite."

# 4. Análisis de resultados

## 4.1. Diagramas de módulos

El sistema se divide en módulos de hardware y de software. La Figura 1 muestra cómo se conectan.

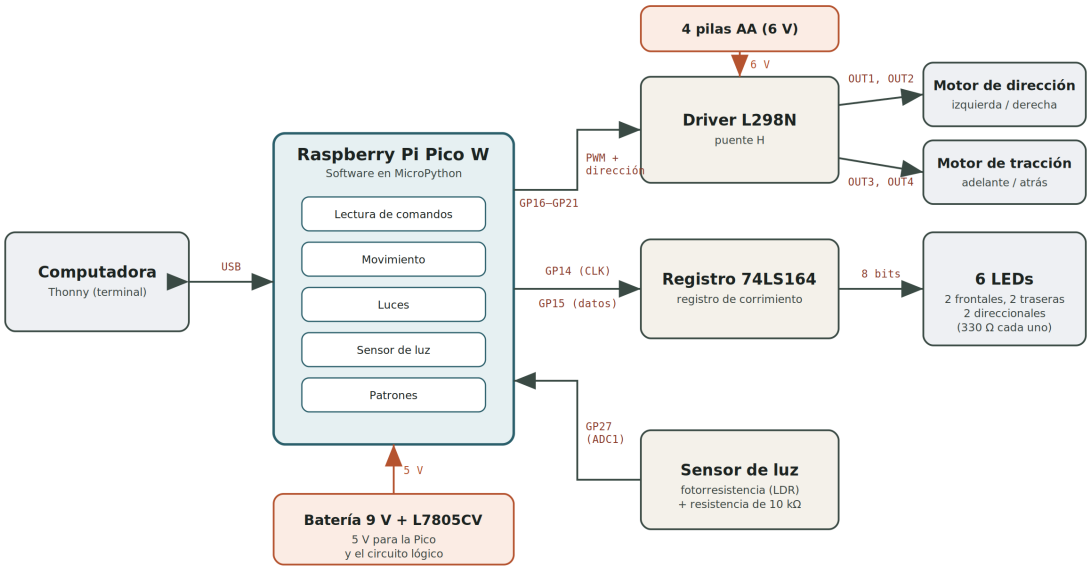



Figura 1: Diagrama de módulos del sistema.

### 4.1.1. Módulos de hardware

La Tabla 1 resume la función de cada módulo. La Tabla 2 muestra a qué pin de la Pico va cada conexión.

| Módulo | Función |
|---|---|
| Raspberry Pi Pico W | Recibe los comandos desde Thonny y genera las señales para los demás componentes. |
| L298N y motores | Puente H que controla los dos motores. El motor de dirección va en OUT1 y OUT2. El de tracción va en OUT3 y OUT4. |
| 74LS164 y LEDs | Registro de corrimiento que controla los seis LEDs con dos pines. Cada LED lleva una resistencia de 330 Ω. |
| Sensor de luz | Divisor de voltaje entre la fotorresistencia y una resistencia de 10 kΩ. La salida va al ADC de la Pico. |
| Alimentación | Cuatro baterías AA en serie (6 V) para los motores. Una batería de 9 V regulada a 5 V con el L7805CV para la Pico y el resto del circuito. Cada una con su interruptor. |

Tabla 1: Módulos de hardware.

| Pin GP | Pin físico | Conexión |
|---|---|---|
| GP14 | 19 | CLK del 74LS164 |
| GP15 | 20 | Entradas A y B del 74LS164 |
| GP16 | 21 | Enable A del L298N (PWM) |
| GP17 | 22 | Input 1 del L298N |
| GP18 | 24 | Input 2 del L298N |
| GP19 | 25 | Enable B del L298N (PWM) |
| GP20 | 26 | Input 3 del L298N |
| GP21 | 27 | Input 4 del L298N |
| GP27 | 32 | Fotorresistencia (ADC1) |

Tabla 2: Conexiones de la Pico W.

#### 4.1.2. Módulos de software

El programa está en un solo archivo de MicroPython. La Tabla 3 muestra cómo se divide.

| Módulo | Funciones y qué hacen |
|---|---|
| Lectura de comandos | `leerLinea` lee lo que se escribe en la terminal sin bloquear el programa. `ejecutarComando` revisa que el comando y el valor sean válidos y llama la función que corresponde. |
| Movimiento | `ponerTraccion` y `ponerDireccion` controlan los motores. `nivelAPotencia` convierte el nivel de `pwm` en potencia. `manejar` mueve el carro durante un tiempo. |
| Luces | `EjeutarSecuencia` manda los 8 bits al registro. `ponerLuces` cambia solo las luces pedidas y guarda el estado. |
| Sensor de luz | `revisarLdr` enciende o apaga las cuatro luces según la lectura. `leerFotoresistencia` sirve para calibrar el umbral. |
| Patrones | `patronCirculo`, `patronInfinito`, `patronZigzag` y `patronEspecial`. |

Tabla 3: Módulos de software.

### 4.2. Plan de pruebas

Cada módulo se probó por separado antes de armar el carro completo. La Tabla 4 resume cada prueba y su resultado.

| **Prueba** | **Procedimiento** | **Resultado esperado** | **Resultado obtenido** |
|---|---|---|---|
| Interruptores | Medir continuidad con el multímetro en las dos posiciones. | Conduce en I y no conduce en O. | El interruptor de la Batería de 6V Conduce en O y no conduce en I se dejo así porque seguía funcionando . El interruptor de la Batería de 9V funciono crretamente |
| Motores | Correr `pruebaCarro.py`. Mueve tracción y dirección en ambos sentidos. | Cada motor gira en los dos sentidos. | Los motores estaban asignados al revés en el código. Se corrigió y ambos giran. |
| Potencia mínima | Probar la tracción con distintos valores de potencia. | Encontrar la potencia más baja que mueve el carro. | El motor no arranca con potencias bajas. Se fijó la mínima en 50000. |
| Pulso de arranque | Dar potencia máxima por 0.2 s antes de cada movimiento. | Que el carro arranque solo. | No resolvió el problema. El motor es muy pequeño para el peso. Se quitó. |
| Luces | Mandar `lf`, `lb`, `ll` y `lr` con 1 y con 0. | Cada comando enciende o apaga solo sus LEDs. | las luces direccionales no encendían correctamente. Se acomodó los cables y se cambio los LEDs quemados. |
| Sensor de luz | Leer la LDR tapada y destapada. | La lectura baja con poca luz. | falla con la resistencia, la fotorresistencia no leía bien la luz del ambiente, se reemplazo la resistencia. |
| Modo `ldr` | Activar `ldr 1` y tapar la LDR. | Se encienden las cuatro luces y se apagan con luz. | los LEDs prendieron correctamente cuando la luz era muy baja o cuando no había luz y se apagan cuando hay vasyante luz. |
| Comandos inválidos | Escribir comandos que no existen y valores fuera de rango. | El programa avisa y no mueve nada. | El programa si avisa cuando un comando que no este en código indica que el comando no es conocido y te recomienda escribir help para ver los comandos disponibles y responde a comandos conocidos com pwm 4. |
| Patrones | Correr `Circle`, `Infinite`, `ZigZag` y `especial`. | Se aprecia cada recorrido. | Se aprecia correctamente los recorridos de los patrones pero el piso y la fricción puede afectar un poco. |

Tabla 4: Plan de pruebas.

## 4.3. Reglas de grupo

1. Reunirse al menos 2 veces por semana para revisar avances.
2. Probar todo cambio en el carro antes de integrarlo y guardar una copia del programa que funciona.
3. Avisar con anticipación si alguien no puede cumplir una fecha.

## 4.4. Roles y fechas de actividades y entregables

| **Integrante** | **Rol** |
|---|---|
| Juan Manuel Bonilla Arce | Software: programa de comandos en MicroPython (lectura de comandos, movimiento, luces, sensor de luz y patrones). Apoyo en el armado y las pruebas. |
| Randall Andrés Mora Zamora | Hardware: armado del circuito, montaje en el carro y reemplazo de componentes. Apoyo en el programa y en la documentación. |

Tabla 5: Roles del grupo.

| **Fecha** | **Actividad o entregable** | **Responsable** |
|---|---|---|
| 12/10/2026 | Armado del circuito en protoboard | Randall |
| 19/10/2026 | Pruebas de módulos por separado | Ambos |
| 21/10/2026 | Programa de comandos | Juan |
| 24/10/2026 | Montaje en el carro | Ambos |
| 31/10/2026 | Documentación | Ambos |
| 2/11/2026, 11:00 | Entrega del hardware en el LuTec y de la documentación en Classroom | Ambos |

Tabla 6: Fechas de actividades y entregables.

# 5. Bitácora

| Fecha | Actividad | Responsable |
|---|---|---|
| 12/10/2026 | Armado de partes del circuito: conexión de la batería de 9 V al regulador L7805CV (5 V) y de los motores al L298N. | Randall |
| 19/10/2026 | Revisión de las hojas de datos del 74LS04, 74LS164 y de los interruptores KCD11. | Ambos |
| 19/10/2026 | Primera prueba de motores, luces y LDR con `pruebaCarro.py`. | Ambos |
| 21/10/2026 | Corrección de los pines de los motores. Los de tracción y dirección estaban invertidos. | Juan |
| 21/10/2026 | Prueba de potencia mínima y del pulso de arranque. Se concluye que el motor es pequeño para el peso. | Juan |
| 21/10/2026 | Programa de comandos completo con movimiento, luces y LDR. | Juan |
| 24/10/2026 | Montaje completo del carro y reemplazo de componentes dañados. Prueba de las luces, en la que se detectaron fallas en las direccionales y lecturas incorrectas de la fotorresistencia. | Ambos |
| 28/10/2026 | Cambio de un LED de direccional y de la resistencia de 10k del divisor de voltaje, que afectaba la medición de la fotorresistencia. | Randall |
| 31/10/2026 | Prueba final del carro completo (patrones, código y LEDs) antes de la entrega. | Randall |

Tabla 7: Bitácora del proyecto.

# Referencias


[1] A. Villegas Lemus, Milton y Barahona Maroto, "Formula e ces tec. energy saving and telemetry. proyecto parte i." Fundamentos de Sistemas Computacionales, Escuela de Ingeniería en Computadores, Instituto Tecnológico de Costa Rica, 2026.

[2] Raspberry Pi Ltd, "Raspberry pi pico W datasheet." https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf. Consultado el 23 de octubre de 2026.

[3] STMicroelectronics, "L298 dual full-bridge driver." https://www.st.com/resource/en/datasheet/l298.pdf. Consultado el 23 de octubre de 2026.

[4] Texas Instruments, "SN54164, SN54LS164, SN74164, SN74LS164: 8-bit parallel-out serial shift registers." https://www.ti.com/lit/ds/symlink/sn74ls164.pdf, 1988.